GUIA DE ESTUDO · FLUTTER

Temas, Formulários e Navegação

Estrutura de projeto, ThemeData com Material 3, layout, formulários com validação, manipulação de strings, classes de modelo e passagem de dados entre telas.

1. Estrutura de um projeto Flutter
2. Widgets: Stateless, Stateful e `const`
3. Cores
4. Tema centralizado (`ThemeData`)
5. Layout e widgets de tela
6. Formulários
7. Strings e números em Dart
8. Classes de modelo em Dart
9. Navegação entre telas
10. Boas práticas e qualidade de código
11. Comandos úteis
12. Revisão rápida

1. Estrutura de um projeto Flutter

1.1 Organização em arquivos

Todo o código Dart do app fica em `lib/`. Separar responsabilidades em arquivos e pastas deixa o projeto legível:

```
lib/
├── main.dart              → main() e MaterialApp
├── theme.dart             → constantes de cor e ThemeData
├── models/
│   └── pessoa.dart        → classes de dados (modelos)
└── pages/
    ├── cadastro_page.dart → uma tela por arquivo
    └── detalhe_page.dart
```

- Nomes de arquivo em `snake_case` (`cadastro_page.dart`); nomes de classe em `PascalCase` (`CadastroPage`).
- Importe outros arquivos do projeto com caminho relativo:

```dart
import 'package:flutter/material.dart';   // widgets Material
import 'package:flutter/services.dart';   // inputFormatters, teclado

import '../models/pessoa.dart';           // sobe uma pasta e entra em models/
import 'detalhe_page.dart';               // mesmo diretório
```

1.2 `main()` e `MaterialApp`

`main()` chama `runApp()` com o widget raiz. O `MaterialApp` define o tema e a primeira tela (`home`):

```dart
import 'package:flutter/material.dart';

import 'pages/cadastro_page.dart';
import 'theme.dart';

void main() {
  runApp(const MeuApp());
}

class MeuApp extends StatelessWidget {
  const MeuApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Meu App',
      debugShowCheckedModeBanner: false, // remove a faixa "DEBUG"
      theme: appTheme,                   // tema claro (definido em theme.dart)
      darkTheme: appDarkTheme,           // tema escuro (opcional)
      themeMode: ThemeMode.system,       // segue o modo do aparelho
      home: const CadastroPage(),        // primeira tela
    );
```

```
  }
}
```

2. Widgets: Stateless, Stateful e `const`

2.1 StatelessWidget

Não guarda estado mutável. Tudo o que exibe vem do construtor. Ideal para telas que apenas **mostram** dados recebidos.

```dart
class DetalhePage extends StatelessWidget {
  final Pessoa pessoa;                              // dado recebido

  const DetalhePage({super.key, required this.pessoa});

  @override
  Widget build(BuildContext context) {
    return Scaffold(/* ... */);
  }
}
```

2.2 StatefulWidget

Usado quando a tela tem estado que vive enquanto ela existe (controllers, chave do formulário, variáveis que mudam com `setState`). São duas classes: o widget e o seu `State`.

```dart
class CadastroPage extends StatefulWidget {
  const CadastroPage({super.key});

  @override
  State<CadastroPage> createState() => _CadastroPageState();
}

class _CadastroPageState extends State<CadastroPage> {
  final _nomeController = TextEditingController();

  @override
  void dispose() {
    _nomeController.dispose(); // libera recursos quando a tela sai da árvore
    super.dispose();           // sempre por último
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(/* ... */);
  }
}
```

Ciclo de vida essencial

Método	Quando roda	Uso típico
`initState()`	uma vez, ao criar o State	inicializar algo que dependa do widget
`build()`	a cada reconstrução	montar a interface
`setState(() { ... })`	chamado por você	alterar variável e redesenhar a tela
`dispose()`	uma vez, ao destruir o State	liberar controllers, listeners, timers

2.3 `const`

- Construtor `const` + campos `final` = widget imutável que o Flutter pode reaproveitar sem reconstruir.
- Use `const` em widgets cujos argumentos são todos constantes: `const Text('Olá')`, `const SizedBox(height: 16)`, `const Icon(Icons.person)`, `const EdgeInsets.all(24)`.
- Se um widget pai é `const`, os filhos já são implicitamente constantes (não precisa repetir `const` dentro).
- Não dá para usar `const` quando há valor calculado em tempo de execução (ex.: `Text(pessoa.nome)`, `Theme.of(context)`).
- `super.key` no construtor repassa a chave para a superclasse — o analisador exige isso em widgets públicos.
- Prefixo `_` torna classes, métodos e variáveis **privados** ao arquivo (`_formKey`, `_LinhaDado`).

3. Cores

- Uma cor hexadecimal `#RRGGBB` vira `Color(0xFFRRGGBB)` — o `FF` inicial é a opacidade (totalmente opaca).
  - `#00897B` → `Color(0xFF00897B)`
- Cores prontas: `Colors.white`, `Colors.black`, `Colors.grey`, `Colors.teal`...
- Declare as cores como **constantes de nível superior** no arquivo de tema e reutilize-as:

```
const corPrimaria = Color(0xFF00897B);
const corFundo    = Color(0xFFE0F2F1);
const corTitulo   = Color(0xFF004D40);
const corErro     = Color(0xFFC62828);
```

4. Tema centralizado (`ThemeData`)

4.1 A ideia

Toda a aparência (cores, fontes, bordas, botões) é configurada **uma vez** em um `ThemeData`. Os widgets não recebem cores nem tamanhos fixos: eles **herdam** do tema automaticamente ou consultam o tema com `Theme.of(context)`.

- **Errado:** `AppBar(backgroundColor: Color(0xFF00897B))` repetido em cada tela, `Text('x', style: TextStyle(fontSize: 26, color: ...))` espalhado pelo código.
- **Certo:** `AppBar(title: Text('Cadastro'))` e `Text('x', style: Theme.of(context).textTheme.headlineMedium)`.

4.2 Material 3 e `ColorScheme.fromSeed`

`ColorScheme.fromSeed` gera uma paleta inteira e harmônica (primary, secondary, surface, error...) a partir de **uma cor-semente**. Como a cor gerada para `primary` pode ficar levemente diferente da semente, passe `primary` explicitamente para fixar a cor exata:

```dart
final ThemeData appTheme = ThemeData(
  useMaterial3: true,
  colorScheme: ColorScheme.fromSeed(
    seedColor: corPrimaria,  // gera a paleta
    primary: corPrimaria,    // garante a cor exata na primary
  ),
  scaffoldBackgroundColor: corFundo, // fundo de todos os Scaffold
  // ... demais temas abaixo
);
```

Cores do esquema ficam acessíveis em qualquer widget: `Theme.of(context).colorScheme.primary`, `.onPrimary`, `.error`, `.surface`.

4.3 `AppBarTheme`

```dart
appBarTheme: const AppBarTheme(
  backgroundColor: corPrimaria,  // fundo da barra
  foregroundColor: Colors.white, // cor de ícones (inclusive a seta de voltar)
  centerTitle: true,             // título centralizado
  titleTextStyle: TextStyle(
    fontSize: 22,
    fontWeight: FontWeight.bold,
    color: Colors.white,         // cor do texto do título
  ),
),
```

> Quando `titleTextStyle` é definido, a cor do título vem dele — por isso a cor precisa estar no `TextStyle`, não só em `foregroundColor`.

4.4 `TextTheme` — tamanhos, cores e pesos de texto

O `TextTheme` tem estilos nomeados. Você define os que precisa e os consome nos widgets:

Estilo	Uso comum
`headlineLarge / Medium / Small`	títulos grandes de tela
`titleLarge / Medium / Small`	títulos de seção, saudações, cabeçalhos
`bodyLarge / Medium / Small`	textos comuns (`bodyLarge` é o texto digitado nos campos)
`labelLarge / Medium / Small`	textos de botões e rótulos pequenos

```
textTheme: const TextTheme(
  headlineMedium: TextStyle(
    fontSize: 26,
    fontWeight: FontWeight.bold,
    color: corTitulo,
  ),
  titleMedium: TextStyle(
    fontSize: 18,
    fontWeight: FontWeight.w600, // seminegrito
    color: corTitulo,
  ),
  bodyLarge: TextStyle(fontSize: 17, color: Color(0xFF212121)),
  bodyMedium: TextStyle(fontSize: 14, color: Color(0xFF546E7A)),
),
```

Pesos de fonte (`FontWeight`)

Descrição	Valor
normal	`FontWeight.normal` (= `w400`)
médio	`FontWeight.w500`
seminegrito	`FontWeight.w600`
negrito	`FontWeight.bold` (= `w700`)

- `fontSize` é em pixels lógicos — equivale ao "sp" das especificações de design (`fontSize: 18` = 18 sp).
- O texto que o usuário digita em um `TextFormField` usa `bodyLarge` por padrão; por isso configurar `bodyLarge` muda o texto digitado.

Consumindo o estilo:

```
final textTheme = Theme.of(context).textTheme;

Text('Título da tela', style: textTheme.headlineMedium),
Text('Subtítulo',      style: textTheme.bodyMedium),
Text('Olá, Ana!',      style: textTheme.titleMedium),
```

Para um ajuste pontual sem perder o resto do estilo, use `copyWith`: `textTheme.bodyMedium?.copyWith(fontStyle: FontStyle.italic)`.

4.5 `InputDecorationTheme` — aparência dos campos

Define borda, rótulo, ícones e mensagens de erro de **todos** os `TextFormField`:

```dart
inputDecorationTheme: InputDecorationTheme(
  filled: true,
  fillColor: Colors.white,
  labelStyle: const TextStyle(fontSize: 15, color: Color(0xFF455A64)), // rótulo
  prefixIconColor: corPrimaria,                                        // ícone à esquerda
  errorStyle: const TextStyle(fontSize: 13, color: corErro),          // mensagem de erro
  border: OutlineInputBorder(borderRadius: BorderRadius.circular(10)),
  enabledBorder: OutlineInputBorder(                                   // sem foco
    borderRadius: BorderRadius.circular(10),
    borderSide: const BorderSide(color: Colors.grey),
  ),
  focusedBorder: OutlineInputBorder(                                   // com foco
    borderRadius: BorderRadius.circular(10),
    borderSide: const BorderSide(color: corPrimaria, width: 2),
  ),
  errorBorder: OutlineInputBorder(                                     // com erro
    borderRadius: BorderRadius.circular(10),
    borderSide: const BorderSide(color: corErro),
  ),
  focusedErrorBorder: OutlineInputBorder(                              // erro + foco
    borderRadius: BorderRadius.circular(10),
    borderSide: const BorderSide(color: corErro, width: 2),
  ),
),
```

Propriedade	Controla
`border`	borda base (usada quando as específicas não são definidas)
`enabledBorder` / `focusedBorder`	borda sem foco / com foco
`errorBorder` / `focusedErrorBorder`	borda quando o campo tem erro
`labelStyle` / `floatingLabelStyle`	rótulo dentro do campo / rótulo flutuando acima
`hintStyle`	texto de exemplo (hint)
`prefixIconColor` / `suffixIconColor`	cor dos ícones
`errorStyle`	tamanho e cor da mensagem de erro
`contentPadding`	espaçamento interno

> `OutlineInputBorder` desenha a caixa em volta; `UnderlineInputBorder` desenha só a linha inferior. Espessura com `BorderSide(width: 2)`, arredondamento com `BorderRadius.circular(10)`.

4.6 Temas de botões

Cada tipo de botão tem seu tema; o estilo é criado com o `styleFrom` do próprio botão:

```dart
elevatedButtonTheme: ElevatedButtonThemeData(
  style: ElevatedButton.styleFrom(
    backgroundColor: corPrimaria,              // fundo
    foregroundColor: Colors.white,             // texto e ícone
    minimumSize: const Size.fromHeight(50),    // altura 50 e largura máxima
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
```

```
    textStyle: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
  ),
),
outlinedButtonTheme: OutlinedButtonThemeData(
  style: OutlinedButton.styleFrom(
    foregroundColor: corPrimaria,                    // texto
    side: const BorderSide(color: corPrimaria),      // borda
    minimumSize: const Size.fromHeight(50),
    shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(10)),
    textStyle: const TextStyle(fontSize: 16, fontWeight: FontWeight.bold),
  ),
),
```

Botão	Tema	Visual
`ElevatedButton`	`elevatedButtonTheme`	preenchido, ação principal
`OutlinedButton`	`outlinedButtonTheme`	só borda, ação secundária
`TextButton`	`textButtonTheme`	só texto
`FilledButton`	`filledButtonTheme`	preenchido sem sombra

- `Size.fromHeight(50)` = altura 50 e largura infinita (o botão ocupa toda a largura disponível).
- A cor do texto vem de `foregroundColor`; a cor/tamanho/peso da fonte em `textStyle` (a cor dentro de `textStyle` é ignorada — use `foregroundColor`).

4.7 `CardThemeData`

```
cardTheme: CardThemeData(
  color: Colors.white,
  elevation: 3,                                                    // sombra
  shape: RoundedRectangleBorder(borderRadius: BorderRadius.circular(16)), // cantos
  margin: const EdgeInsets.all(8),                                 // opcional
),
```

> Nas versões atuais do Flutter, `ThemeData.cardTheme` recebe `CardThemeData` (não `CardTheme`).

4.8 Tema escuro e `ThemeMode`

Crie um segundo `ThemeData` com `brightness: Brightness.dark` no `ColorScheme` e registre-o no `MaterialApp`:

```
final ThemeData appDarkTheme = ThemeData(
  useMaterial3: true,
  colorScheme: ColorScheme.fromSeed(
    seedColor: corPrimaria,
    brightness: Brightness.dark, // gera a paleta escura
  ),
);

// no MaterialApp:
theme: appTheme,
```

```
darkTheme: appDarkTheme,
themeMode: ThemeMode.system, // .light / .dark / .system (segue o aparelho)
```

5. Layout e widgets de tela

5.1 Estrutura básica de uma tela

```
return Scaffold(
  appBar: AppBar(title: const Text('Cadastro')), // cores vêm do appBarTheme
  body: SingleChildScrollView(                   // permite rolar (teclado aberto)
    padding: const EdgeInsets.all(24),
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.stretch, // filhos ocupam toda a largura
      children: [
        Text('Título', style: textTheme.headlineMedium),
        const SizedBox(height: 8),                    // espaço vertical
        Text('Subtítulo', style: textTheme.bodyMedium),
        const SizedBox(height: 24),
        // campos e botões...
      ],
    ),
  ),
);
```

Widget	Função
`Scaffold`	estrutura da tela (appBar, body, floatingActionButton)
`AppBar`	barra superior; mostra a **seta de voltar automaticamente** quando há tela anterior na pilha
`SingleChildScrollView`	torna o conteúdo rolável, evita *overflow* quando o teclado aparece
`Padding` / `padding:`	margem interna
`Column` / `Row`	empilha filhos na vertical / horizontal
`SizedBox(height: x)`	espaçador; `SizedBox(width: double.infinity, child: ...)` estica o filho
`Center`	centraliza o filho na vertical e na horizontal
`Expanded`	ocupa o espaço restante dentro de `Row` / `Column`

5.2 Eixos de `Column` e `Row`

- `mainAxisAlignment` → alinhamento no eixo principal (vertical na Column): `start`, `center`, `end`, `spaceBetween`, `spaceEvenly`.
- `crossAxisAlignment` → eixo cruzado (horizontal na Column): `start`, `center`, `end`, `stretch`.
- `mainAxisSize: MainAxisSize.min` → a Column ocupa só a altura dos filhos (útil para centralizar o bloco).

Botões com largura total — qualquer uma das opções:

1. `crossAxisAlignment: CrossAxisAlignment.stretch` na Column;
2. `minimumSize: Size.fromHeight(50)` no tema do botão;
3. `SizedBox(width: double.infinity, child: ElevatedButton(...))`.

5.3 Centralizar conteúdo na tela

```
body: Center(                          // centraliza vertical e horizontalmente
  child: SingleChildScrollView(        // rola se não couber
    padding: const EdgeInsets.all(24),
    child: Column(
      mainAxisSize: MainAxisSize.min,   // altura = conteúdo, para o Center funcionar
      children: [
        Icon(Icons.check_circle, size: 80, color: theme.colorScheme.primary),
        const SizedBox(height: 16),
        Text('Tudo certo!', style: textTheme.headlineMedium),
        Text('Olá, ${pessoa.primeiroNome}!', style: textTheme.titleMedium),
      ],
    ),
  ),
),
```

5.4 `Card`, `ListTile` e `Icon`

`Card` agrupa informações numa superfície elevada (estilo vindo do `cardTheme`). `ListTile` monta uma linha com ícone, rótulo e valor:

```
Card(
  child: Padding(
    padding: const EdgeInsets.all(16),
    child: Column(
      children: [
        ListTile(
          leading: Icon(Icons.person_outline, color: theme.colorScheme.primary),
          title: Text('Nome', style: textTheme.bodyMedium),      // rótulo
          subtitle: Text(pessoa.nome, style: textTheme.bodyLarge), // valor
        ),
        ListTile(
          leading: Icon(Icons.cake_outlined, color: theme.colorScheme.primary),
          title: Text('Idade', style: textTheme.bodyMedium),
          subtitle: Text('${pessoa.idade} anos', style: textTheme.bodyLarge),
        ),
      ],
    ),
  ),
),
```

Para não repetir código, extraia a linha para um widget próprio:

```
class _LinhaDado extends StatelessWidget {
  final IconData icone;
  final String rotulo;
  final String valor;
```

```
  const _LinhaDado({required this.icone, required this.rotulo, required this.valor});

  @override
  Widget build(BuildContext context) {
    final theme = Theme.of(context);
    return ListTile(
      leading: Icon(icone, color: theme.colorScheme.primary),
      title: Text(rotulo, style: theme.textTheme.bodyMedium),
      subtitle: Text(valor, style: theme.textTheme.bodyLarge),
    );
  }
}
```

- Ícones: `Icon(Icons.nome_do_icone)`; variantes terminam em `_outlined`, `_outline`, `_rounded`. Tamanho padrão 24.
- Ícone colorido com a cor primária: `color: Theme.of(context).colorScheme.primary` (nunca uma cor fixa).

6. Formulários

6.1 As peças

Peça	Papel
`Form`	agrupa os campos e permite validar/resetar todos de uma vez
`GlobalKey<FormState>`	chave que dá acesso ao estado do `Form` (`validate()`, `reset()`)
`TextFormField`	campo de texto que participa do `Form` e aceita `validator`
`TextEditingController`	lê e altera o texto de um campo (`.text`, `.clear()`)

> `TextField` **não** tem `validator` e não participa do `Form`. Em formulários com validação, use `TextFormField`.

6.2 Montando o formulário

```
class _CadastroPageState extends State<CadastroPage> {
  final _formKey = GlobalKey<FormState>();
  final _nomeController = TextEditingController();
  final _emailController = TextEditingController();
  final _idadeController = TextEditingController();

  @override
  void dispose() {
    _nomeController.dispose();
    _emailController.dispose();
    _idadeController.dispose();
    super.dispose();
  }
```

```dart
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Cadastro')),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(24),
        child: Form(
          key: _formKey,                    // liga a chave ao Form
          child: Column(
            crossAxisAlignment: CrossAxisAlignment.stretch,
            children: [
              TextFormField(
                controller: _nomeController,
                textCapitalization: TextCapitalization.words,
                textInputAction: TextInputAction.next,
                decoration: const InputDecoration(
                  labelText: 'Nome completo',
                  prefixIcon: Icon(Icons.person_outline),
                ),
                validator: _validarNome,
              ),
              const SizedBox(height: 16),
              // ... outros campos
            ],
          ),
        ),
      ),
    );
  }
}
```

6.3 `InputDecoration` — o que aparece no campo

Propriedade	Exibe
`labelText`	rótulo (sobe para a borda quando o campo recebe foco)
`hintText`	texto de exemplo em cinza, some ao digitar (ex.: `'usuario@dominio.com'`)
`prefixIcon` / `suffixIcon`	ícone à esquerda / à direita
`helperText`	texto de ajuda abaixo do campo
`counterText`	contador de caracteres (`''` esconde)

> A decoração de cada campo só define **conteúdo** (rótulo, ícone, hint). Cores, bordas e fontes ficam no `inputDecorationTheme`.

6.4 Teclado e comportamento de digitação

`keyboardType` — qual teclado aparece no celular:

Valor	Teclado
`TextInputType.text`	texto comum (padrão)
`TextInputType.emailAddress`	com `@` e `.` em destaque
`TextInputType.phone`	numérico de telefone
`TextInputType.number`	numérico
`TextInputType.multiline`	várias linhas

`textCapitalization` — capitaliza automaticamente:

Valor	Efeito
`TextCapitalization.words`	Primeira Letra De Cada Palavra (nomes)
`TextCapitalization.sentences`	primeira letra de cada frase
`TextCapitalization.characters`	TUDO MAIÚSCULO

`textInputAction` — o botão de ação do teclado:

- `TextInputAction.next` → "próximo": passa o foco para o campo seguinte automaticamente.
- `TextInputAction.done` → "concluir": usado no último campo; combine com `onFieldSubmitted: (_) => _salvar()` para enviar ao pressionar Enter.

> No navegador de desktop o `keyboardType` não muda nada visível (o teclado é físico), mas ele continua sendo exigido no código. Já os `inputFormatters` funcionam em qualquer plataforma.

6.5 `inputFormatters` — restringir o que pode ser digitado

Filtram a entrada **enquanto o usuário digita** (letras nem chegam a aparecer). Precisam de `import 'package:flutter/services.dart';`.

```
TextFormField(
  controller: _idadeController,
  keyboardType: TextInputType.number,
  inputFormatters: [
    FilteringTextInputFormatter.digitsOnly,   // só 0–9
    LengthLimitingTextInputFormatter(3),      // no máximo 3 caracteres
  ],
  decoration: const InputDecoration(
    labelText: 'Idade',
    prefixIcon: Icon(Icons.cake_outlined),
  ),
  validator: _validarIdade,
),
```

Formatter	Efeito
`FilteringTextInputFormatter.digitsOnly`	aceita só dígitos
`FilteringTextInputFormatter.allow(RegExp(r'[a-zA-Z]'))`	aceita só o que casa com a regex
`FilteringTextInputFormatter.deny(RegExp(r'\s'))`	bloqueia o que casa com a regex (aqui, espaços)
`LengthLimitingTextInputFormatter(n)`	limita a `n` caracteres

> Alternativa ao limitador: `maxLength: n` no `TextFormField` (mas ele mostra um contador "0/n" embaixo do campo).

Formatter restringe, validator confere. O formatter impede caracteres errados; o validator verifica regras de conteúdo (tamanho mínimo, faixa numérica, formato).

6.6 Validadores

Um `validator` é uma função que recebe o texto do campo e devolve:

- **`null`** → campo válido;
- **uma `String`** → campo inválido; o texto devolvido é a mensagem de erro exibida abaixo do campo.

```dart
String? _validarAlgo(String? valor) {
  final texto = valor?.trim() ?? '';           // 1. normaliza (null vira '')
  if (texto.isEmpty) return 'Informe o campo.'; // 2. vazio primeiro
  if (/* regra violada */ false) {
    return 'Mensagem da regra.';                // 3. demais regras, na ordem
  }
  return null;                                  // 4. passou em tudo
}
```

- Verifique **vazio primeiro**, depois as regras específicas — cada `return` encerra a função, então só a primeira mensagem aplicável aparece.
- Copie as mensagens **exatamente** como especificado (acentos, ponto final).
- O validator pode ser escrito inline (`validator: (valor) { ... }`) ou como método (`validator: _validarNome`).

Técnicas de validação reutilizáveis

```dart
// Campo obrigatório
if (texto.isEmpty) return 'Informe o valor.';

// Quantidade mínima de palavras (ex.: exigir pelo menos 2)
final palavras = texto.split(RegExp(r'\s+')); // divide por 1 ou mais espaços
if (palavras.length < 2) return 'Digite ao menos duas palavras.';

// Formato via expressão regular
final regex = RegExp(r'^[\w.+-]+@[\w-]+(\.[\w-]+)+$'); // padrão "algo@dominio.ext"
if (!regex.hasMatch(texto)) return 'Formato inválido.';

// Quantidade exata (ou um conjunto) de caracteres
```

```dart
if (texto.length != 8) return 'Deve ter 8 caracteres.';
if (texto.length != 8 && texto.length != 9) return 'Deve ter 8 ou 9 caracteres.';

// Faixa numérica
final numero = int.tryParse(texto);           // null se não for número
if (numero == null || numero < 18 || numero > 99) {
  return 'Valor deve estar entre 18 e 99.';
}
```

Lendo uma regex de formato `^[\w.+-]+@[\w-]+(\.[\w-]+)+$`:

Trecho	Significa
`^` e `$`	início e fim do texto (casa o texto inteiro)
`[\w.+-]+`	uma ou mais letras, dígitos, `_`, `.`, `+` ou `-`
`@`	o caractere `@` literal
`[\w-]+`	domínio (letras, dígitos, `-`)
`(\.[\w-]+)+`	um ou mais grupos `.algo` (ex.: `.com`, `.com.br`)

> Verificar apenas `texto.contains('@')` **não** é validação de formato — aceita `@`, `a@`, `@b`.
> Use `r'...'` (raw string) nas regex para que `\` não precise ser escapado.

6.7 Enviar (`validate`) e limpar (`reset`)

```dart
void _salvar() {
  // validate() roda todos os validators, exibe as mensagens e retorna true/false
  if (!_formKey.currentState!.validate()) return; // inválido: fica na tela

  // válido: monta o objeto com os dados tratados e navega
  final pessoa = Pessoa(
    nome: _nomeController.text.trim().split(RegExp(r'\s+')).join(' '),
    email: _emailController.text.trim(),
    idade: int.parse(_idadeController.text),
  );

  Navigator.push(
    context,
    MaterialPageRoute(builder: (context) => DetalhePage(pessoa: pessoa)),
  );
}

void _limpar() {
  _nomeController.clear();          // apaga o texto de cada campo
  _emailController.clear();
  _idadeController.clear();
  _formKey.currentState!.reset();   // remove as mensagens de erro exibidas
}
```

```dart
ElevatedButton(onPressed: _salvar, child: const Text('Salvar')),
const SizedBox(height: 12),
OutlinedButton(onPressed: _limpar, child: const Text('Limpar')),
```

- `currentState!` — o `!` afirma que o estado não é nulo (o `Form` já está na tela).
- Só `clear()` apaga o texto mas **mantém** as mensagens de erro; só `reset()` remove erros. Use os dois.
- `onPressed: _salvar` passa a função (sem parênteses). `onPressed: _salvar()` executaria na hora — errado.
- `onPressed: null` deixa o botão desabilitado.

Validação automática (opcional) — `autovalidateMode` no `Form` ou no campo:

Valor	Quando valida
`AutovalidateMode.disabled`	só ao chamar `validate()` (padrão)
`AutovalidateMode.onUserInteraction`	a cada edição, depois que o usuário mexe no campo
`AutovalidateMode.always`	o tempo todo

7. Strings e números em Dart

7.1 Operações mais usadas

Operação	Exemplo	Resultado
Remover espaços nas pontas	`' Ana '.trim()`	`'Ana'`
Dividir por espaços (1 ou mais)	`'Ana Maria'.split(RegExp(r'\s+'))`	`['Ana', 'Maria']`
Juntar lista	`['Ana', 'Maria'].join(' ')`	`'Ana Maria'`
Primeiro / último elemento	`lista.first` / `lista.last`	—
Tamanho	`'abc'.length`	`3`
Vazio?	`''.isEmpty` / `'a'.isNotEmpty`	`true` / `true`
Trecho por posição	`'abcdef'.substring(0, 2)`	`'ab'` (fim exclusivo)
Do índice até o fim	`'abcdef'.substring(2)`	`'cdef'`
Remover não-dígitos	`'(31) 9'.replaceAll(RegExp(r'\D'), '')`	`'319'`
Texto → int (lança erro se inválido)	`int.parse('42')`	`42`
Texto → int seguro	`int.tryParse('abc')`	`null`
Interpolação	`'Olá, $nome!'` / `'${pessoa.idade} anos'`	—

7.2 Normalizar espaços internos

`trim()` remove espaços só das pontas. Para também transformar espaços repetidos internos em um só:

```dart
final normalizado = texto.trim().split(RegExp(r'\s+')).join(' ');
// '  Ana    Maria  Souza ' → 'Ana Maria Souza'
```

7.3 Montar uma máscara com `substring`

Formatar um texto é recortá-lo em pedaços por posição e juntá-los com interpolação. Contar posições **a partir do fim** funciona para textos de tamanhos diferentes:

```dart
// CEP: '30140071' → '30140-071'
final cep = '30140071';
final formatado = '${cep.substring(0, 5)}-${cep.substring(5)}';

// Separar os 4 últimos caracteres, qualquer que seja o tamanho:
final corte = texto.length - 4;
final resultado = '${texto.substring(0, corte)}-${texto.substring(corte)}';
// '12345678' → '1234-5678'    '123456789' → '12345-6789'

// Prefixo entre parênteses + resto:
final comPrefixo = '(${texto.substring(0, 2)}) ${texto.substring(2)}';
```

> Teste mentalmente com cada tamanho possível de entrada. Lembre que em `substring(inicio, fim)` o índice `fim` **não** entra.

8. Classes de modelo em Dart

8.1 Classe imutável

Um modelo agrupa os dados de uma entidade em um único objeto, que pode ser passado entre telas.

```dart
class Pessoa {
  final String nome;      // final = não muda depois de criado
  final String email;
  final int idade;        // tipo correto: int, não String

  const Pessoa({          // construtor const com parâmetros nomeados
    required this.nome,   // required = obrigatório informar
    required this.email,
    required this.idade,
  });
}

// Criação — parâmetros nomeados deixam claro o que é cada valor:
const p = Pessoa(nome: 'Ana Souza', email: 'ana@x.com', idade: 20);
```

Conceito	Significado
`final`	atributo atribuído uma única vez
construtor `const`	permite criar instâncias constantes; exige todos os campos `final`
`{ ... }` nos parâmetros	parâmetros **nomeados** (`Pessoa(nome: ...)`)
`required`	parâmetro nomeado obrigatório
`this.campo`	atribui o argumento direto ao atributo

8.2 Getters — valores calculados

Um getter é uma "propriedade calculada": acessada como campo (sem parênteses), mas computada a partir dos outros dados.

```dart
class Pessoa {
  // ... campos e construtor

  // primeira palavra do nome
  String get primeiroNome => nome.split(' ').first;

  // iniciais: 'Ana Souza' → 'AS'
  String get iniciais =>
      nome.split(' ').map((parte) => parte[0].toUpperCase()).join();

  // getter com corpo de várias linhas
  String get emailMascarado {
    final partes = email.split('@');
    return '${partes[0][0]}***@${partes[1]}';
  }
}

// uso:
Text('Olá, ${pessoa.primeiroNome}!')
```

- Sintaxe curta: `Tipo get nome => expressão;`
- Sintaxe longa: `Tipo get nome { ...; return valor; }`

9. Navegação entre telas

9.1 A pilha de rotas

O `Navigator` mantém uma **pilha** de telas:

- `Navigator.push` **empilha** uma nova tela por cima. A tela de baixo continua existindo (com seu `State`, controllers e textos digitados).
- `Navigator.pop` **desempilha** a tela atual e revela a anterior, exatamente como estava.

```
push(Detalhe)            pop()
┌────────────┐
│  Detalhe   │  ←topo
├────────────┤
│  Cadastro  │          ┌────────────┐
└────────────┘          │  Cadastro  │  ← volta com os campos preenchidos
                        └────────────┘
```

9.2 Ir para outra tela passando um objeto

Os dados vão pelo **construtor** da tela de destino:

```dart
// Tela de origem
Navigator.push(
  context,
```

```
  MaterialPageRoute(
    builder: (context) => DetalhePage(pessoa: pessoa), // objeto inteiro
  ),
);
```

```
// Tela de destino
class DetalhePage extends StatelessWidget {
  final Pessoa pessoa;

  const DetalhePage({super.key, required this.pessoa});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: const Text('Detalhes')), // seta de voltar automática
      body: Center(child: Text(pessoa.nome)),
    );
  }
}
```

- Passe **o objeto** (`pessoa: pessoa`), não vários valores soltos (`nome: ..., email: ..., idade: ...`).
- **Não** use variáveis globais ou `static` para "compartilhar" dados entre telas — o construtor é o canal correto.

9.3 Voltar

```
OutlinedButton(
  onPressed: () => Navigator.pop(context), // desempilha: volta à tela anterior
  child: const Text('Voltar'),
),
```

- Como a tela anterior nunca saiu da pilha, os campos continuam preenchidos ao voltar.
- **Erro comum:** usar `Navigator.push(context, MaterialPageRoute(builder: (_) => const CadastroPage()))` para "voltar". Isso cria uma tela **nova** (campos vazios) e a pilha cresce a cada ida e volta.
- A seta da `AppBar` já faz `pop` automaticamente — não precisa programar.
- `onPressed: () => Navigator.pop(context)` usa uma função anônima porque `pop` precisa de argumento.

9.4 Rotas e retorno de valor (complementar)

```
// pop pode devolver um valor para quem fez o push:
final resultado = await Navigator.push<String>(
  context,
  MaterialPageRoute(builder: (_) => const OutraPage()),
);
// na OutraPage: Navigator.pop(context, 'valor');

// substituir a tela atual (sem possibilidade de voltar):
Navigator.pushReplacement(context, MaterialPageRoute(builder: (_) => const Home()));
```

10. Boas práticas e qualidade de código

- **`dispose()`**: todo `TextEditingController` criado no `State` deve ser liberado em `dispose()`, terminando com `super.dispose()`.
- **`const`**: em construtores de widgets e modelos, e em widgets com argumentos fixos (`const Text`, `const SizedBox`, `const EdgeInsets`, `const InputDecoration(...)` quando tudo dentro é constante).
- **Nomes significativos**: `_nomeController`, `_validarEmail`, `_salvar`, `primeiroNome` — nada de `c1`, `f()`, `x`.
- **Tema, não estilo fixo**: nenhum `Color(...)` ou `fontSize` direto nos widgets das telas; tudo via `Theme.of(context)`.
- **Tipos corretos**: número como `int` (`int.parse`), texto tratado com `trim()`.
- **Imports sem uso** e variáveis não usadas geram avisos — remova.

Avisos comuns do `flutter analyze`

Aviso	Correção
`prefer_const_constructors`	adicionar `const` antes do construtor
`prefer_const_literals_to_create_immutables`	`const [...]` em listas fixas
`use_key_in_widget_constructors`	adicionar `{super.key}` ao construtor
`unused_import`	apagar o import
`prefer_final_fields`	declarar o campo como `final`
`use_build_context_synchronously`	não usar `context` após `await` sem checar `mounted`

11. Comandos úteis

```
flutter create nome_app          # cria um projeto novo
cd nome_app
flutter run -d chrome            # executa no navegador
flutter analyze                  # lista erros e avisos do código
flutter clean && flutter pub get # resolve problemas de build/cache
```

Com o app rodando no terminal:

Tecla	Ação
`r`	*hot reload* — aplica mudanças mantendo o estado
`R`	*hot restart* — reinicia o app; necessário após alterar `main()`, `initState` ou variáveis globais (como o `appTheme`)
`q`	encerra

Compactar o projeto (macOS/Linux):

```
zip -r prova_NomeSobrenome.zip nome_app -x "nome_app/build/*" "nome_app/.dart_tool/*"
```

12. Revisão rápida

- ☐ Arquivos separados: `main.dart`, `theme.dart`, `models/`, `pages/`.
- ☐ `MaterialApp` com `theme`, `home` (e opcionalmente `darkTheme` + `themeMode`).
- ☐ `ThemeData` com `useMaterial3`, `ColorScheme.fromSeed`, `scaffoldBackgroundColor`, `appBarTheme`, `textTheme`, `inputDecorationTheme`, temas de botões e `cardTheme`.
- ☐ Widgets consomem o tema via `Theme.of(context)` — sem cores/tamanhos fixos.
- ☐ `Form` + `GlobalKey<FormState>` + `TextFormField` + `TextEditingController`.
- ☐ Cada campo: `prefixIcon`, `labelText`, `keyboardType`, `textInputAction`, `inputFormatters` e `validator` quando aplicável.
- ☐ Validators retornam `null` (ok) ou a mensagem exata; vazio verificado primeiro.
- ☐ Enviar: `validate()` → montar objeto com dados tratados (`trim`, `int.parse`) → `Navigator.push`.
- ☐ Limpar: `clear()` em cada controller + `reset()` no form.
- ☐ Modelo imutável: campos `final`, construtor `const`, parâmetros `required`, getters.
- ☐ Tela de destino recebe o objeto por `required this.objeto`; volta com `Navigator.pop`.
- ☐ `dispose()` dos controllers, `const` onde possível, `flutter analyze` sem avisos.